PORTFOLIO · HBM DIGITAL DESIGN (RTL DESIGN)

Mini DRAM Controller

Command-level FR-FCFS · 행 / 열 명령 버스 · Per-bank Refresh · RFM — Spec · RTL · 검증 · PPA

지원 직무	SK hynix · HBM Digital Design (RTL Design)
지원자	권용현 · 010-6408-8646 · kyongh0331@naver.com
저장소	`github.com/yhkwon-git/mini-dram-controller` · `v4 commit ab9d522` · `RTL 44a0bb3`
사용 도구	SystemVerilog · Verilator 5.052 · Yosys 0.52 · sv2v · OpenSTA 3.1 (Nangate45) · Python

p.2–26의 v4 수치는 RTL commit ab9d522 기준(44a0bb3은 ModelSim 캐스트와 q_empty 출력만 더했고 directed 로그가 같다). p.12–13 파형은 44a0bb3의 VCD, 버전 비교는 각 버전 commit 재실행, 분석 스크립트 322632f, HSL 숫자는 HSL 3d596a4.

00 Summary

v0 → v4: 단일 FSM에서 행 / 열 명령 버스 · Per-bank Refresh · RFM까지

같은 DRAM 모델 위에서 컨트롤러를 다섯 번 설계하고, 전환마다 latency · 처리량 · 면적 · 타이밍을 같은 TB와 같은 합성 흐름으로 비교

버전	구조와 바뀐 점	포화 row hit RD	fmax · 면적
v0	단일 FSM: 요청 하나를 PRE · ACT · RD/WR 순서로 처리	–	1,182 MHz · 1,208 μm²
v1	깊이 4 큐 + 뱅크 엔진 ×4 + 사이클당 명령 1개 arbiter	19.9%	343 MHz · 9,692 μm²
v2	명령 단위 FR-FCFS(age matrix), tREFI refresh + PREA, RD 4개 파이프라인, 타이밍 7종 추가	66.7%	357 MHz · 13,478 μm²
v3	읽기 · 쓰기 큐 8 + 8, posted write · 포워딩, 쓰기 묶음, page 정책 3 · 주소 매핑 4	66.7%	337 MHz · 18,600 μm²
v4	행 · 열 명령 버스, 뱅크 그룹(tCCD · tRRD · tWTR S/L), per-bank refresh, RFM	**80.7%**	**348 MHz** · 18,912 μm²

포화: 요청 간격 0, back-pressure 없음, seed 1, 5,000건. v1 처리량은 2eabd33, 면적 · 타이밍은 fa08330. v4부터 장치에 뱅크 그룹이 있어 같은 장치의 v3 방식 빌드(bus1_refab)는 63.3%다. 면적 · fmax는 Nangate45 typical, mem_ctrl 단독.

80.7%

포화 row hit 읽기, 데이터 버스 사용률 · 같은 장치의 v3 방식 63.3% (+27%)

1,200 / 1,200

랜덤 회귀 PASS · 600만 요청 · 빌드 16 · 규칙 위반 0 · 기능 bin 32/32

98.7 · 98.2%

컨트롤러 Line · Branch 커버리지 (Toggle 93.1%), directed + 랜덤

348 MHz

Nangate45 최소 주기 2.874 ns · 18,912 μm² · FF 1,616 (v3 337 MHz)

가장 큰 손잡이: per-bank refresh

all-bank로 바꾸면 포화 평균 −16%, 면적 차이는 2%. 버스 분리와 함께일 때 row hit 63.3 → 80.7%

타이밍 클로저: 208 → 337 MHz (v3)

수락 경로를 스케줄러에서 떼고, 후보 집합을 합치고, 선택을 one-hot 그대로 유지. v4는 기능을 더하고도 348 MHz

검증: 독립 체커 + 랜덤 회귀

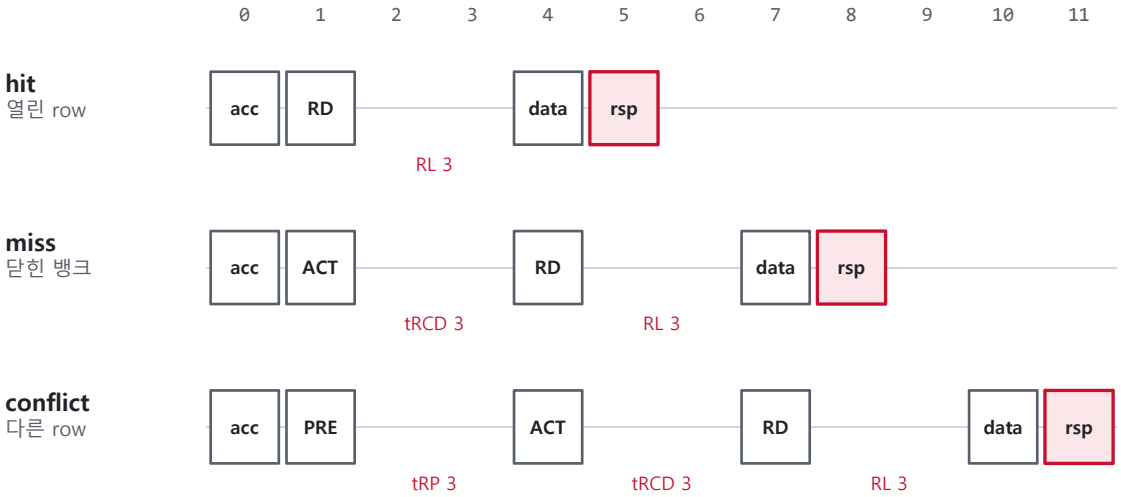
DRAM 핀만 보는 bind 체커(타이밍 18 · 프로토콜 10종)와 class 랜덤 TB. 회귀 1,200회 위반 0, TB race 1건 디버그

읽는 순서 p.3–21 v4 상세 (스펙, 구조, RTL, 파형, 검증, 디버그, trade-off, HSL 연결, 버전별 latency) → p.22–26 v2 – v4 비교 (구조, 검증 인프라, 성능, 트래픽 × 기능 분석, PPA)

컨트롤러의 일: 뱅크 상태가 만드는 대기를 다른 요청으로 가리기

같은 뱅크에서도 page hit RD 5사이클, row conflict RD 11사이클 · 타이밍은 자체 축소 모델(JEDEC 값 아님) · v4, commit ab9d522

요청 하나의 명령 순서 (v4, 수락 엣지 = 0, 그 뒤 엣지마다)



- **수락 → 첫 명령 1사이클**: 수락 엣지에 큐에 들어가고, 다음 엣지에 스케줄러가 고른 명령이 나간다
- **RL 3 뒤 데이터, 1사이클 뒤 응답**: 데이터를 응답 버퍼에 넣은 다음 엣지에 rsp_valid. 쓰기와 포워딩된 읽기는 수락 다음 엣지에 응답(1사이클)
- **PRE · ACT의 대기 6사이클**이 conflict를 hit의 두 배 넘게 만든다. 그 사이에 다른 뱅크의 RD/WR · ACT를 넣는 것이 처리량을 정한다

v4가 다루는 범위

- host_if → mem_ctrl(읽기 큐 8 + 쓰기 큐 8, 스케줄러, 뱅크 ×4, 채널, refresh, 응답) → dram_model(뱅크 그룹 2 × 뱅크 2). 단일 클럭, 비동기 리셋
- DRAM 쪽: 행 버스(ACT · PRE · PREA · REF · REFpb)와 열 버스(RD · WR)가 따로, 고정 지연 RL 3 · WL 1. busy 신호 없이 모든 규칙을 컨트롤러가 센다
- 호스트 쪽: valid/ready, 응답은 요청 순서, 쓰기는 posted
- 범위 밖: 실제 PHY, multi-rank, ECC, 사인오프 PPA

HBM에서의 위치

- 표준 HBM(HBM3E까지)은 컨트롤러가 호스트 SoC에 있다. HBM4부터 컨트롤러를 base die에 두는 custom HBM 구조가 나왔고, High-Speed Link Lite가 이 RTL을 그 자리에 넣었다(p.20)

이 장 (p.3–21)	내용
p.4–6	스펙(주소 · 명령 · 인터페이스 · 타이밍), 요청의 생애와 FF 구성
p.7–13	RTL 블록 다섯, v4 파형 두 개
p.14–21	검증 · 디버그 · trade-off · HSL 연결 · 버전별 latency

스펙 ①: 주소 매핑, 명령 집합, DRAM 쪽 · 호스트 쪽 인터페이스

상수는 rtl/dram_pkg.sv 한 곳, 같은 값을 docs/spec.md가 문서로 관리 · 4 bank × 256 row × 16 col × 32 bit (64 KiB) · 14 bit 워드 주소

주소 매핑 (ADDR_MAP, 모두 전단사)

매핑	배치 (MSB → LSB)	효과
XOR (기본)	row | bank ^ row[1:0] | col	16 word씩 한 row, 64 word stride도 뱅크를 돈다
row:bank:col	row | bank | col	16 word가 한 row, 다음은 다음 뱅크
row:col:bank	row | col | bank	연속 word가 뱅크를 돈다
bank:row:col (v2)	bank | row | col	뱅크마다 주소 공간 1/4

명령 (dram_cmd_e, 3 bit)

버스	명령 (코드)	동작
행	ACT 001 · PRE 100 · PREA 110	row 열기 · 한 뱅크 닫기 · 모두 닫기
행	REF 101 · REFpb 111	모든 뱅크(뒤 tRFC 12) · 한 뱅크(뒤 tRFCpb 8)
열	RD 010 · WR 011 (+ ap)	열린 row에서 32 bit 버스트 하나. row 필드 없음, ap면 닫음

- **두 버스는 매 엣지 샘플**: 한 엣지에 행 명령 하나와 열 명령 하나가 같이 나갈 수 있다(HBM 방식). 열 버스는 그 뱅크의 열린 row를 쓴다
- **매핑은 TB가 따로 구현**: tb_loc은 비트 슬라이스 대신 나눗셈과 나머지로 같은 매핑을 계산해, RTL 매핑 버그가 짝 없는 명령으로 드러난다

DRAM 쪽 (DFI 형태: 고정 지연, busy 없음)

신호	방향	의미
dram_rcmd · rbank · row	출력	이번 사이클의 행 명령
dram_ccmd · cbank · col · ap	출력	이번 사이클의 열 명령
dram_wdata_en · wdata	출력	WR 뒤 WL(1) 엣지에 쓰기 데이터
dram_data_valid · rdata	입력	RD 뒤 RL(3) 엣지, RD 순서. data_valid로 진행 중 RD를 센다
q_empty	출력	두 큐가 빔 (HSL 레인 BIST가 읽기 전에 기다림)

호스트 쪽 (valid/ready)

규칙	내용
수락	응답 tag 여유(진행 중 ≤ 15). 읽기는 읽기 큐 빈자리(포워딩이면 불필요), 쓰기는 쓰기 큐 빈자리와 읽기 큐에 같은 주소 없음
같은 주소	쓰기 큐에 있는 주소의 읽기는 가장 나중 쓰기로 바로 응답(포워딩). 같은 주소 쓰기는 들어온 순서대로
완료 · 순서	읽기는 데이터가 올 때(포워딩이면 수락 때), 쓰기는 수락 때(posted, rdata 0). 응답은 요청 순서
back-pressure	rsp_ready = 0이면 16-entry 버퍼에서 대기, 진행 중 15개면 수락을 멈춤. rsp_err는 예약(0)

스펙 ②: 타이밍 파라미터와 그것을 지키는 블록

컨트롤러 클럭 사이클 · 간격 = 두 이벤트를 샘플한 엣지 사이의 사이클 수(모델과 체커가 같은 규약) · 자체 축소 값, JEDEC 아님

규칙	값	내용	지키는 블록
tRCD	3	ACT → RD/WR, 같은 뱅크	mc_bank
tRP	3	PRE · PREA · auto-precharge → ACT · REF	mc_bank
tRAS · tRC	6 · 9	ACT → PRE · ACT → ACT, 같은 뱅크	mc_bank
tRTP · tWR	2 · 2	RD → PRE · WR → PRE ≥ WL + 1 + tWR = 4	mc_bank
tCCD_S / L	1 / 2	RD/WR → RD/WR, 다른 / 같은 그룹	mc_chan
tRRD_S / L	2 / 3	ACT → ACT (REFpb도 tRRD_S에서는 ACT)	mc_chan
tWTR_S / L	2 / 4	WR → RD, 다른 / 같은 그룹	mc_chan
tRTW · tFAW	4 · 8	RD → WR · 8사이클 창에 ACT 4개까지	mc_chan
tRFC · tRFCpb	12 · 8	REF 뒤 모든 명령 · REFpb 뒤 그 뱅크의 ACT	mc_chan · mc_bank
tREFI · tREFIpb	64 · 16	뱅크마다 tREFI당 한 번 · per-bank는 16마다 REFpb	mc_refresh
RL · WL	3 · 1	RD → 읽기 데이터 · WR → 쓰기 데이터 (정확히)	mc_resp
RAA_IMT / MMT	4 / 12	RFM 시점 · 이 값에 닿으면 ACT 금지	mc_bank · mc_sched

규칙끼리의 관계

- 데이터 버스 충돌 없음: RD 데이터는 RD + 3, WR 데이터는 WR + 1. tRTW 4가 WR 데이터를 앞 RD 데이터 뒤로, tWTR이 RD를 앞 쓰기 데이터 뒤로 민다
- WR → PRE 4 = WL 1 + 데이터 1 + tWR 2. write recovery는 데이터 뒤부터 센다
- auto-precharge: 장치가 PRE가 합법이 되는 첫 엣지(tRAS, tRTP, write recovery)에 스스로 닫고, 다음 ACT · REF는 그 엣지 + tRP
- RFM 규모: tRC 9라 한 tREFI(64)에 뱅크당 ACT는 최대 7개, RAA_IMT의 약 두 배 (실제 장치와 같은 비율)
- refresh 비용이 실제보다 크다(tRFC / tREFI = 12 / 64). 절대값보다 버전과 정책 사이의 비교에 쓴다

뱅크 그룹 (BG = 뱅크 번호의 MSB)

그룹	뱅크	같은 그룹 안의 연속 명령
BG 0	b0 · b1	열 2사이클(tCCD_L) · ACT 3(tRRD_L) · WR → RD 4(tWTR_L)
BG 1	b2 · b3	연속 주소를 한 그룹에 모으는 매핑은 연속 읽기가 50%에서 막힘 (p.24)

구조: 요청 하나의 생애와 블록별 플롭

블록 그림과 한 사이클의 결정 표는 p.22 · 여기서는 요청이 어느 블록을 언제 지나는지와 FF 1,616개가 어디에 있는지 (commit ab9d522)

요청의 생애 (읽기: 1 → 7, 쓰기는 1에서 응답, 6에서 데이터)

단계	블록	하는 일
1 수락	mem_ctrl	주소 매핑 → 같은 주소 비교(match) → req_ready. 쓰기 · 포워딩된 읽기는 이 엣지에 응답 버퍼로
2 큐	mc_queue ×2	빈 슬롯에 push, age matrix 열 하나 기록, 같은 주소 쓰기 의존(q_dep)
3 다음 명령	mc_sched	hit → RD/WR, miss → ACT, conflict → PRE와 준비 여부(mc_bank · mc_chan · RD 슬롯)
4 행 명령	행 버스	ACT · PRE는 요청을 큐에 둔 채 나간다. refresh · RFM 명령이 먼저
5 열 명령	열 버스	RD/WR가 나가는 엣지에 큐에서 pop (one-hot 그대로)
6 데이터	mc_resp	RD의 tag를 read FIFO(4)에, data_valid마다 하나 꺼냄. WR 데이터는 WL 파이프
7 응답	mc_resp	16-entry 버퍼에 완료 표시, tag_retire 순서로 호스트에

- **모듈**: mem_ctrl이 mc_queue ×2 · mc_xage · mc_wdrain · mc_sched · mc_bank ×4 · mc_chan · mc_refresh · mc_resp를 잇는다. 파라미터 13개(DEPTH, FR_FCFS, CMD_BUS, REF_MODE …)

FF 1,616개 구성 (v4, Yosys stat)

블록	FF	주요 레지스터
쓰기 큐 (mc_queue)	560	q_wdata 256 · q_wait 64 · q_row 64 · q_dep 64 · q_older 56 · q_col 32
응답 (mc_resp)	555	응답 데이터 512 · done 16 · read FIFO 23 · tag 4
읽기 큐 (mc_queue)	276	q_wait 64 · q_row 64 · q_older 56 · q_tag 32 · q_col 32
뱅크 ×4 (mc_bank)	104	open row 32 · 타이밍 카운터 48 · RAA 20 · open 4
채널 (mc_chan)	75	카운터 55 (그룹별 tCCD_L · tRRD_L · tWTR_L 30) · tFAW 창 20
쓰기 파이프 · refresh · 모드	46	쓰기 데이터 33 · refresh 11 · 모드 2

읽는 법

- 데이터를 담는 플롭(응답 512, 쓰기 데이터 256 + 32)이 전체의 50%. 응답 버퍼 하나가 34%다
- q_older의 대각선(자기보다 오래됨)은 상수 0이라 큐마다 64가 아닌 56
- WDRAIN = 1(기본)에서는 두 큐 사이 age(mc_xage)와 bank idle 카운터가 쓰이지 않아 합성에서 빠진다

RTL ①: 요청 큐와 age matrix — 비교 체인 없이 가장 오래된 것 고르기

rtl/mc_queue.sv ×2 (읽기 8 · 쓰기 8) · mc_sched.sv의 oldest_in · 고른 결과는 one-hot 그대로 pop · tag · 데이터 선택까지

rtl/mc_queue.sv · push

```
// q_older[j][i] = 1: j가 i보다 먼저 들어옴
if (push) begin
  q_bank[free_idx] <= push_bank;   // row, col, wdata, tag도
  for (int j = 0; j < DEPTH; j++) begin
    q_older[j][free_idx] <= q_valid[j];  // 있던 것은 모두 더 오래됨
    q_older[free_idx][j] <= 1'b0;
  end
end
// 쓰기 큐(ORDERED = 1): 같은 주소의 앞선 쓰기를 기다림
q_dep[free_idx][j] <= match[j];
match[j] = q_valid[j] && q_bank[j] == push_bank &&
           q_row[j] == push_row && q_col[j] == push_col;
```

rtl/mc_sched.sv · oldest_in (one-hot)

```
for (int i = 0; i < DEPTH; i++) begin
  r[i] = m[i];
  for (int j = 0; j < DEPTH; j++)
    if (j != i && m[j] && old[j][i]) r[i] = 1'b0;
end
// 집합 m 안에서 자기보다 오래된 원소가 없는 하나:
// 엔트리마다 AND-OR 한 단, 인덱스 인코딩 없음
```

왜 age matrix인가

- tag 비교 체인으로 나이를 정한 v2 첫 구현은 151 MHz였다. age matrix로 바꿔 357 MHz (c1b8248)
- push 때 열 하나만 쓴다: 새 엔트리는 모두보다 젊다. pop은 q_valid만 내리고, 행렬의 낡은 비트는 valid가 가린다
- 대각선은 상수 0 → 합성에서 빠져 큐마다 56 FF
- 두 큐 사이 순서는 mc_xage(w_older_r). WDRAIN = 1(기본)에서는 한 번에 한 큐만 후보라 쓰이지 않는다

같은 주소 규칙 (기대값을 정하는 근거)

- 쓰기 → 쓰기: q_dep로 들어온 순서대로 나간다. 앞선 쓰기가 남은 동안은 RD/WR 후보에서 빠진다 (hazard)
- 읽기, 쓰기 큐에 같은 주소: 가장 나중 쓰기의 데이터로 포워딩(p.11). 쓰기, 읽기 큐에 같은 주소: 그 읽기가 나갈 때까지 포트에서 대기
- match는 이번 엣지에 나가는 엔트리도 비교한다: 수락 경로가 스케줄러 결정을 기다리지 않는다(v3 타이밍 클로저, p.26)
- 회귀 1,200회: 포워딩 151,757 · 쓰기 뒤 같은 주소 쓰기 143,018 · 포트에서 기다린 쓰기 470,830, 데이터 모두 일치

RTL ②: 스케줄러 — 요청마다 다음 명령과 준비 여부, 두 버스에 하나씩

rtl/mc_sched.sv · FR-FCFS · 열 버스와 행 버스 후보를 따로 골라 같은 엣지에 낸다 (버스별 우선순위 표는 p.22)

rtl/mc_sched.sv · 요청 (q, i)마다 (q = 0 읽기 큐, 1 쓰기 큐, b = 뱅크, g = 그룹)

```
hit  = q_valid && bank_open[b] && bank_row[b] == q_row;
conf = q_valid && bank_open[b] && bank_row[b] != q_row;
miss = q_valid && !bank_open[b];
col_rdy = live && hit && !hazard && !blocked[b] && bank_col_ok[b] &&
          !starve_bank[b] && (q ? chan_wr_ok[g]
                                : chan_rd_ok[g] && rd_slot_ok);
row_rdy = live && !blocked[b] &&
  (miss && bank_act_ok[b] && !bank_raa_mmt[b] && chan_act_ok[g] ||
   conf && bank_pre_ok[b] && chan_any_ok && !col_bank_rdy[b] &&
           (!guarded || starve_bank[b]));
ccand = |col_rdy ? col_rdy : xcol_rdy;  // 고른 큐, 없으면 다른 큐
rcand = |row_rdy ? row_rdy : prep_rdy;
col_sel = oldest2(ccand);  row_sel = oldest2(rcand);
```

항	뜻
live · blocked	WDRAIN이 고른 큐의 요청 · refresh가 잡은 뱅크
guarded	열린 row를 쓸 요청이 두 큐 어디든 있음(FR-FCFS)
col_bank_rdy	그 뱅크에 열 명령이 준비됨 → PRE를 내지 않음

보호 규칙

- row 보호: 열린 row를 쓸 요청이 남아 있으면 그 뱅크를 닫지 않는다(FR-FCFS). FCFS는 더 오래된 hit만 지킨다
- 같은 뱅크에 두 버스가 겹치지 않게: 열 명령이 준비된 뱅크에는 PRE를 내지 않는다
- starvation: 가장 오래된 후보가 160사이클 넘게 기다리면 oldest-first로, 그 뱅크는 젊은 hit을 멈춘다. 그만큼 기다린 쓰기는 쓰기 모드로
- refresh: 갚을 뱅크는 ACT와 RD/WR를 막고 PRE → REFpb만

page 정책 (PAGE_POLICY)

- open(기본): conflict나 refresh가 닫을 때까지 · close: 같은 row를 쓸 요청이 없으면 RD/WR에 auto-precharge
- timer: 다른 일이 없을 때, 16사이클 쓰이지 않고 기다리는 요청도 없는 row를 PRE

- **회귀 1,200회**: 재정렬 3,386,363 · 행 · 열 명령이 한 엣지에 1,359,346 · 다른 그룹으로 연달아 나간 RD/WR 566,659

RTL ③: 뱅크 · 채널 타이밍 — 남은 사이클을 세는 카운터

rtl/mc_bank.sv ×4 · rtl/mc_chan.sv · 카운터가 0이 된 다음 엣지에 바로 발행 → 체커가 잰 최소 간격이 스펙과 같다 (p.15)

rtl/mc_bank.sv

```
// 카운터 = 명령이 합법이 될 때까지 남은 사이클, 0 = 다음 엣지에 합법
// 이벤트는 (spec - 1) 이상을 넣고, 제약이 겹치면 큰 값이 남는다
assign act_ok = !is_open && act_wait == '0;   // tRP · tRC · tRFCpb
assign col_ok =  is_open && col_wait == '0;   // tRCD
assign pre_ok =  is_open && pre_wait == '0;   // tRAS · tRTP · tWR
if (do_rd) pre_next = hold_for(pre_wait, tRTP);
if (do_wr) pre_next = hold_for(pre_wait, WL + 1 + tWR);
if (do_act) begin
  act_wait <= hold_for(act_wait, tRC);
  col_wait <= hold_for(col_wait, tRCD);
  pre_wait <= hold_for(pre_wait, tRAS);
end
if (do_pre)   act_wait <= hold_for(act_wait, tRP);
if (do_refpb) act_wait <= hold_for(act_wait, tRFCpb);
// auto-precharge: 장치가 닫는 엣지(pre_next + 1) + tRP까지 ACT 대기
```

- **카운터 폭**: 뱅크 4 bit(가장 긴 대기 tRC − 1 = 8), 채널 5 bit(tRFC − 1 = 11). 채널의 짧은 카운터는 상위 비트가 돌지 않아 mc_chan toggle 52%
- **FF**: 뱅크마다 26(open row 8, 카운터 12, RAA 5, open 1) × 4 = 104, 채널 75

rtl/mc_chan.sv · 그룹 g별 준비 신호

```
assign any_ok    = (rfc == '0);     // all-bank REF 뒤 tRFC
assign act_ok[g] = any_ok && rrd_s == '0 && rrd_l[g] == '0
                   && faw_free;
assign rd_ok[g]  = any_ok && ccd_s == '0 && ccd_l[g] == '0
                   && wtr_s == '0 && wtr_l[g] == '0;
assign wr_ok[g]  = any_ok && ccd_s == '0 && ccd_l[g] == '0
                   && rtw == '0;
// tFAW: 슬롯 4개. ACT마다 빈 슬롯에 FAW_LOAD, 하나라도 0이면 ACT 가능
if (do_act) faw_wait[faw_slot] <= FAW_LOAD;
```

규약을 체커와 맞춘 이유

- 체커도 '샘플한 엣지 사이의 사이클 수'로 잰다. 카운터가 끝나는 엣지에 바로 내면 측정 최소 = 스펙이 되어, 한 사이클 늦게 내는 비효율(v1의 handoff +1)도 숫자로 드러난다
- 그룹별 _L 카운터와 채널 공통 _S 카운터를 따로 둬 같은 그룹 · 다른 그룹 규칙을 한 클럭에 함께 본다
- REFpb도 tRRD_S에서는 ACT로 센다. 체커도 같은 규칙으로 잰다

RTL ④: per-bank refresh와 RFM — 한 뱅크만 막고 나머지는 계속

rtl/mc_refresh.sv · mc_bank.sv(RAA) · mc_sched.sv(refresh · RFM 명령) · tREFIpb = tREFI / 4 = 16마다 REFpb 하나, 뱅크를 돌아가며

rtl/mc_refresh.sv

```
assign tick  = (refi == '0);              // 16사이클마다 하나 빚짐
assign quiet = (MODE == REF_BANK) ? !wanted[bank] : idle;
assign start = (owed != '0) && (quiet || owed >= 4'(URGENT));
assign hold  = busy_q || start;  // 그 뱅크: ACT · RD/WR 금지
refi <= tick ? REFI_LOAD : refi - 1'b1;
if (MODE == REF_BANK && do_ref) bank <= bank + 1'b1;
```

refresh 방식 (REF_MODE)

항목	per-bank (기본)	all-bank (v3)
빚	16사이클마다 REFpb 하나, 뱅크 순서대로	64사이클마다 REF 하나
갚는 때	그 뱅크를 기다리는 요청이 없을 때, 4개 밀리면 바로	두 큐가 빌 때, 4개 밀리면 바로
순서	그 뱅크만 PRE → REFpb	모든 뱅크 PREA → REF
뒤	그 뱅크만 tRFCpb 8, 나머지는 계속	모든 명령 tRFC 12 대기

RFM (RowHammer 대응)

- RAA: 뱅크마다 ACT 수. ACT마다 +1, 그 뱅크 refresh마다 −4(RAA_IMT, 0 아래로 안 감)
- 12(RAA_MMT)에 닿은 뱅크는 ACT를 받지 않는다. 열린 row를 쓸 요청이 없어지면 닫고 REFpb를 준다
- RFM_EARLY(기본): RAA 4 이상이고 아무 요청도 쓰지 않는 닫힌 뱅크에는 미리 REFpb
- 비용: FF 20(뱅크당 RAA 5 bit), hotbank 처리량 약 5%(seed 3, 커밋 전 RFM 없는 v4 대비)

측정

항목	값
뱅크당 refresh 간격 (directed)	평균 62.2, 최대 91 엣지 (tREFI 64)
밀린 refresh 최대	1 (v3 all-bank 4)
RAA 최대 (directed)	8 (RFM 시점 4, 한도 12)
RAA 한도 도달 (랜덤 1,200회)	131,879회, 한도 넘은 ACT 0
REFpb · 그 엣지에 다른 뱅크 RD/WR	1,674,822 · 111,515

RTL ⑤: 수락 · 포워딩 · 응답 — posted write와 요청 순서 응답

rtl/mem_ctrl.sv(수락 · 포워딩) · mc_wdrain.sv(쓰기 묶음) · mc_resp.sv(tag · read FIFO · 쓰기 파이프 · 응답 버퍼)

rtl/mem_ctrl.sv · 수락과 포워딩

```
assign fwd       = !req_write && |q_match[1];  // 쓰기 큐에 같은 주소
assign collide   =  req_write && |q_match[0];  // 읽기 큐에 같은 주소
assign req_ready = tag_free && (fwd || q_has_free[req_write])
                   && !collide;
assign q_push    = {acc && req_write, acc && !req_write && !fwd};
// 포워딩 데이터: 같은 주소 쓰기 중 가장 나중 것 (age로 AND-OR)
fwd_sel[i] = q_match[1][i];
for (j ...) if (j != i && q_match[1][j] && q_older[1][i][j])
  fwd_sel[i] = 1'b0;
fwd_data |= {DATA_BITS{fwd_sel[i]}} & q_wdata[1][i];
```

쓰기 묶음 (mc_wdrain, wmode는 레지스터)

모드	언제
읽기 (기본)	읽기 먼저
쓰기	쓰기 큐가 6(WR_HI)에 닿으면 2(WR_LO)까지, 읽기가 없을 때, 160사이클 기다린 쓰기가 있을 때
(공통)	금지가 아니라 우선순위: 고른 큐가 못 쓰는 슬롯은 다른 큐가 쓴다

응답 쪽 (mc_resp)

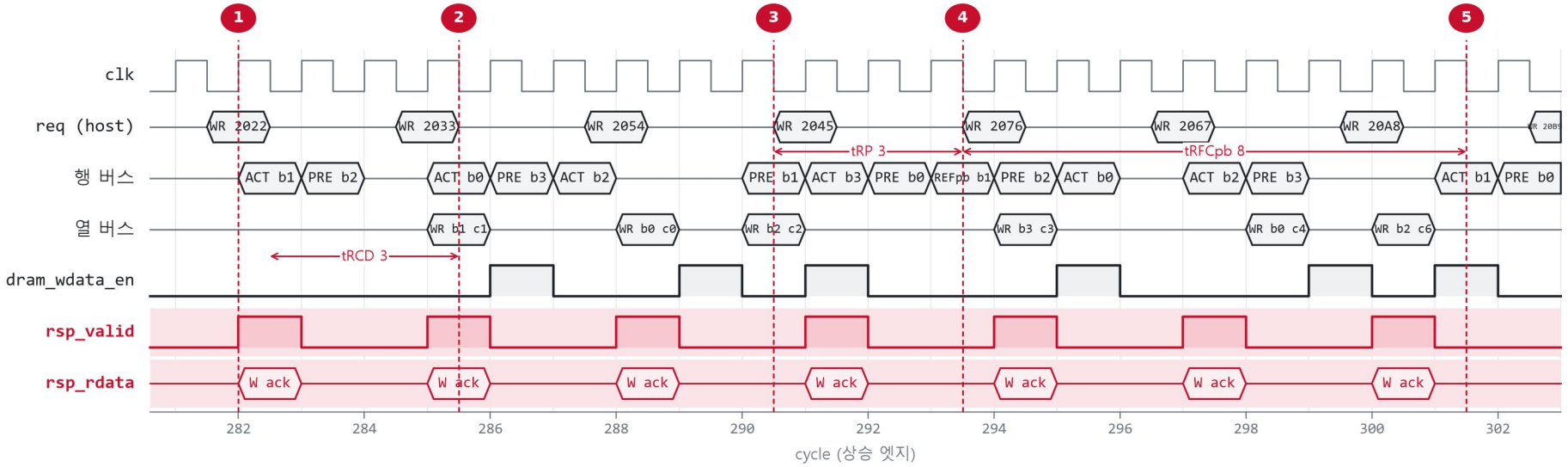
- tag 4 bit, 진행 중 최대 15개(tag_free = tag_alloc + 1 ≠ tag_retire). v1 tag wrap-around를 고친 규칙이 그대로 남았다(p.17)
- RD는 tag를 read FIFO(깊이 RD_PIPE = 4)에 넣고 data_valid마다 하나 꺼낸다. PHY가 데이터를 늦게 줘도 순서만 맞으면 된다(HSL, p.20)
- WR 데이터는 WL 단 파이프가 wdata_en으로 낸다
- 완료된 응답은 16 × 32b 버퍼에서 tag_retire 순서로 나간다. 버퍼가 FF의 34%

회귀에서 본 횟수 (1,200회, 기능 bin 합계)

상황	합계
쓰기 큐에서 포워딩된 읽기	151,757
읽기 큐에 같은 주소가 있어 포트에서 기다린 쓰기	470,830
tag 한도(15)에서 요청 대기 (사이클)	13,517,082
WR 4개 이상 연속 (쓰기 묶음)	319,245

v4 파형 ①: 쓰기 흐름 — 행 · 열 버스 동시 발행과 per-bank refresh

make sim WAVES=1의 Verilator VCD(RTL 44a0bb3, directed 로그는 ab9d522와 같음) · 시나리오 5 앞의 쓰기 20개 중 cycle 281–302 · XOR 매핑



bank 1 refresh (엣지)

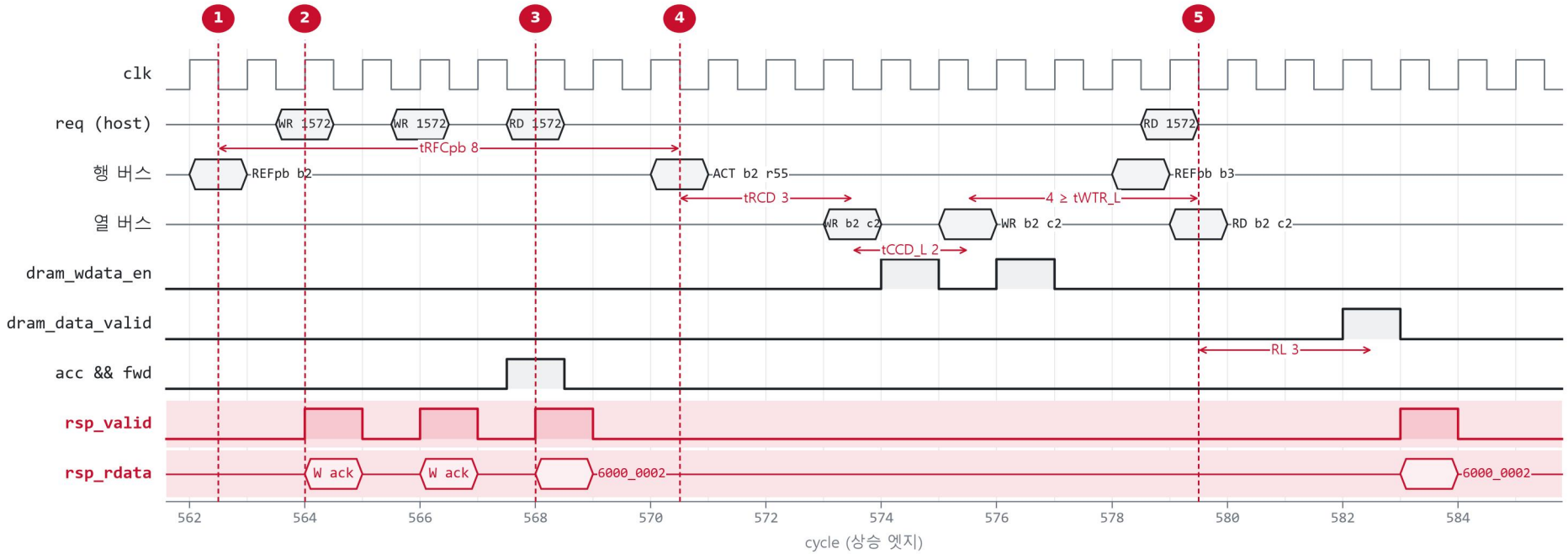
명령	엣지
ACT b1 r80	283
WR b1 c1	286
PRE b1	291
REFpb b1	294
ACT b1 r81	302

- **REFpb는 16사이클마다 다음 뱅크**: b0 278 → b1 294

- ① 엣지 282에 WR 2022를 수락하고 283에 W ack (posted). 데이터는 WR와 함께 나중에 나간다
- ② 엣지 286: 행 버스 ACT b0와 열 버스 WR b1 c1이 같은 엣지. 버스가 하나면(v3) 한 사이클씩 줄을 선다
- ③ 엣지 291: bank 1의 refresh 차례. PRE b1(WR 286 + 4, ACT 283 + tRAS 6 뒤)과 WR b2 c2가 같은 엣지
- ④ 엣지 294: REFpb b1 = PRE + tRP 3. 그동안 b0 · b2 · b3는 ACT · PRE · WR를 계속 받는다
- ⑤ 엣지 302: ACT b1 r81 = REFpb + tRFCpb 8. refresh가 막은 것은 bank 1 하나뿐 (all-bank면 모든 row를 닫고 약 25사이클 멈춤)
- **간격**: ACT b1 → WR b1 = tRCD 3, WR 다음 엣지에 wdata_en (WL 1). 육각형은 그 명령이 버스에 있는 사이클, 샘플은 오른쪽 상승 엣지

v4 파형 ②: refresh 직후 같은 주소 — posted write, 포워딩, 같은 뱅크 타이밍

시나리오 6, cycle 562–585 (RTL 44a0bb3) · 1572 = bank 2 · row 55 · col 2 · 그 뱅크의 REFpb 직후에 시작해 ACT 시점이 고정된다



간격 (규칙)

구간	사이클
REFpb → ACT	8 tRFCpb
ACT → WR	3 tRCD
WR → WR	2 tCCD_L
WR → 데이터	1 WL
WR → RD	4 ≥ tWTR_L
RD → 데이터	3 RL

- **acc && fwd**: 수락하는 엣지에 포워딩된 읽기

- ① 563 REFpb b2: bank 2는 tRFCpb 8 동안 ACT를 받지 않는다
- ② WR 1572 둘을 564 · 566에 수락, 다음 엣지에 W ack. 두 쓰기는 아직 쓰기 큐에 있다
- ③ 568 RD 1572: 쓰기 큐에 같은 주소 → 나중 쓰기의 데이터 6000_0002로 569에 응답, 열 버스에 RD 없음
- ④ 571 ACT b2 r55(563 + 8) → 574 WR(tRCD 3) → 576 WR(같은 뱅크 tCCD_L 2). 같은 주소 두 쓰기는 들어온 순서대로(q_dep)
- ⑤ 579에 다시 읽으면 쓰기가 나간 뒤라 580에 RD b2 c2(576 WR 뒤 4클럭으로 tWTR_L 충족), 583 data_valid(RL 3), 584 응답: hit RD 5사이클

검증 ①: directed 시나리오, 핀만 보는 체커, 응답 판정 규칙

모든 시뮬레이션에 bind 체커 두 개 · directed 6개(make sim)와 랜덤 TB(make regress, p.23)가 같은 체커를 쓴다 · commit ab9d522

directed 시나리오 (tb/tb_mem_ctrl.sv)

#	시나리오	확인 내용
1	page hit WR → RD	쓰기가 DRAM에 나간 뒤 read-back
2	page miss (bank 0 row 변경)	PRE → ACT 경로, 새 row와 이전 row의 데이터
3	multi-bank + conflict	bank 0 row가 바뀐 뒤에도 bank 1 데이터 유지
4	4-bank write burst	ACT는 나갔고 WR는 아직인 뱅크가 동시에 2개 이상
5	응답 back-pressure	rsp_ready 150사이클 0, read 20건(tag 16 초과): 20/20 순서 · 데이터
6	포워딩	같은 주소 쓰기 2건 뒤 읽기: 나중 쓰기 데이터, DRAM RD 0건

```
make sim 출력 (발췌, Verilator 5.052)
[TB] peak banks opening at once=2
[TB] back-pressure OK (20/20 in order, 15 REF during the test)
[TB] forwarded RD OK (0x60000002)
[TB] access_err count=0 OK
[TCHK] PASS: 0 violation(s), 0 waived
[HCHK] PASS: controller violations 0, host violations 0
[TB] PASS — hit/miss/multi-bank/burst/back-pressure/forwarding OK
```

체커 구현의 세부 (p.23 요약에 없는 것)

- 한 엣지의 두 버스 명령은 열 명령부터 본다: 같은 뱅크 PRE + RD는 tRTP 위반, ACT + RD는 '닫힌 뱅크에 RD'로 잡힌다
- 밀린 refresh는 뱅크마다 '지난 tREFI 주기 수 − 받은 refresh 수'. 한도 8은 DDR4의 REF postpone 개수를 빌렸다
- RAA도 체커가 따로 센다(ACT +1, 그 뱅크 refresh −4). 12에 닿은 뱅크에 ACT가 오면 위반
- --assert 없이 빌드하면 assertion이 꺼진 채 통과할 수 있어, 시작할 때 켜졌는지 보고 꺼져 있으면 실패

응답 판정 (랜덤 TB의 scoreboard)

- posted write: 응답은 바로 오지만 그 WR가 핀에 나가는지 따로 본다. 마지막 응답 뒤 2,000사이클 안에 안 나간 쓰기는 실패
- 포워딩: 수락 엣지의 핀 활동까지 기록한 하강 엣지에 예측한다. 그 엣지에 쓰기 큐에 있던 쓰기가 있는데 RD가 나가면 실패, 없는데 RD 없이 응답하면 실패
- 열 버스에는 row가 없다: 다른 row에 간 RD/WR는 TB의 짝짓기(열린 row로 계산한 주소)가 잡는다

검증 ②: 타이밍 18종 · 프로토콜 10종, 측정 최소가 스펙과 같다

dram_timing_checker(bind) · make sim 로그(commit ab9d522) · 측정 최소 = 스펙: 카운터가 끝나는 엣지에 바로 냄 · 랜덤 1,200회도 위반 0

규칙	스펙	측정 최소	검사한 간격
tRCD ACT → RD/WR	3	3	61
tRP PRE · PREA · AP → ACT · REF	3	3	53
tRAS ACT → PRE · PREA	6	6	53
tRC ACT → ACT, 같은 뱅크	9	9	50
tRTP RD → PRE	2	2	29
tWR WR → PRE (WL + 1 + tWR)	4	4	28
tRRD_S ACT · REFpb → ACT · REFpb	2	2	88
tRRD_L ACT → ACT, 같은 그룹	3	3	50
tFAW ACT 4개 창	8	10	50
tCCD_S RD/WR → RD/WR	1	2	60
tCCD_L 같은 그룹	2	2	59
tWTR_S WR → RD	2	4	31
tWTR_L WR → RD, 같은 그룹	4	4	31
tRTW RD → WR	4	4	29
tRFCpb REFpb → 그 뱅크 ACT · REFpb	8	8	34
tRFC REF → 다음 명령	12	–	0
RL · WL 데이터 지연 (정확히)	3 · 1	3 · 1	31 · 30

프로토콜 10종 (위반 0)

- 열린 뱅크에 ACT · 닫힌 뱅크에 RD/WR · 열린 row가 있는 REF · 열린 뱅크에 REFpb
- RD 없는 읽기 데이터 · 오지 않은 읽기 데이터 · WR 없는 쓰기 데이터 · 오지 않은 쓰기 데이터
- 뱅크당 밀린 refresh > 8 · RAA 12 이상인 뱅크에 ACT

make sim 끝의 버스 집계

```
[TCHK] row bus: ACT 54, PRE 53, PREA 0, REF 0, REFpb 37;
       column bus: RD 31, WR 30; both on one edge 27
[TCHK] refresh interval per bank avg 62.2, max 91 edges
[TCHK] peak RAA 8 (RFM due at 4, limit 12)
```

- **측정 최소 > 스펙**(tFAW 10, tCCD_S 2, tWTR_S 4)은 directed 순서에서 그보다 붙은 경우가 없었다는 뜻. 랜덤 회귀가 더 많은 간격을 본다
- **tRFC 0회**: 기본 빌드는 per-bank라 REF를 쓰지 않는다. refab · bus1_refab 회귀에서 검사
- **waiver 0**: v1은 refresh 두 항목을 알려진 문제로 따로 셌고, v2에서 고쳐 끔

검증 ③: 기능 커버리지 32 bin과 남은 구멍

랜덤 TB mc_coverage의 bin 합계 · 회귀 1,200회 · 코드 커버리지 표는 p.23 · 구멍은 tools/cov_summary.py --holes로 찾음 (commit ab9d522)

bin	합계	맞힌 실행
read hit · miss · conflict	894,831 · 734,816 · 1,213,055	1200 · 1200 · 961
write hit · miss · conflict	1,102,895 · 689,846 · 1,212,800	1200 · 1200 · 966
15개 in flight (tag 한도)	786,858	1192
재정렬 (뒤 요청의 RD/WR가 먼저)	3,386,363	1200
DRAM 버스에 RD 2개 · 3개 동시	942,730 · 50,449	1140 · 793
행 · 열 명령이 한 엣지에	1,359,346	1080
다른 그룹으로 연달아 RD/WR	566,659	1185
REFpb · 그 엣지에 다른 뱅크 RD/WR	1,674,822 · 111,515	1160 · 1053
RFM REFpb · RAA 12 도달	709,148 · 131,879	979 · 800
auto-precharge RD/WR (close)	211,907	60
REF 앞 PREA (all-bank 빌드)	40,059	120
응답이 남은 채 · rsp_ready 0인 채 refresh	1,703,953 · 663,627	1200 · 1200

- **맞힌 실행이 적은 bin**은 그 기능이 없는 빌드 때문: 두 버스 bin이 없는 120회는 bus1 계열, PREA는 all-bank 두 빌드만. conflict 없는 실행은 대부분 rowhit 패턴

모듈별 코드 커버리지 (p.23 표에 없는 두 모듈, directed → + 랜덤)

모듈	Line	Branch	Toggle
mc_chan (그룹 타이밍)	100 → 100%	96.2 → 100%	46.3 → 52.1%
mc_refresh	100 → 100%	83.3 → 91.7%	89.1 → 94.0%

- **mc_chan toggle 52%**: 모든 채널 카운터가 tRFC − 1 = 11을 세는 5 bit라, 짧은 카운터의 상위 비트가 돌지 않는다. 기능 구멍이 아니라 폭의 문제
- **mc_refresh branch 91.7%**: 밀린 refresh 카운터의 포화(15). REF_URGENT 4에서 갚기 시작해 설계상 닿지 않는다
- **나머지 구멍**(older() 분기, WL > 1 쓰기 단, dram_model 오류 경로)은 p.23 끝줄에 있다

닫힌 구멍 (v1 → v4)

- out-of-order dispatch와 write-hit: 재정렬 3,386,363 · write hit 1,102,895, 응답 순서 유지
- 늦은 REF · PREA 없는 REF(v1): v2부터 tREFI마다 갚고 먼저 닫음, 둘 다 위반으로 셈 → 0
- starvation · FCFS 경로: starve24 · fcfs · fcfs_nodrain 빌드가 회귀와 커버리지에서 돈다

디버그 ①: 응답 back-pressure에서 tag wrap-around (v1)

coverage.md에 gap으로 적은 back-pressure를 실험 TB로 재현하다 찾음 · 수정 fa08330, 회귀 7e2d4b5 · 같은 15개 규칙이 v4의 tag_free

증상 → 원인 → 수정

- **증상**: rsp_ready를 150사이클 내린 채 read 20건 → 풀린 뒤 앞 4건의 데이터가 틀리고 뒤 4건은 응답이 오지 않음(16/20)
- **원인**: req_ready가 큐의 빈 슬롯만 봤다. 큐는 dispatch로 계속 비어 응답이 막혀도 요청을 받았고, 4-bit tag_alloc이 16을 돌아 반환 전 tag를 다시 할당 → 17번째 read가 첫 read의 버퍼 엔트리를 덮어씀
- **수정 (fa08330)**: 처리 중인 요청이 15개면 받지 않는다
- **회귀 (7e2d4b5)**: 시나리오 5로 고정. 수정 전 RTL은 첫 응답에 0x50000010(17번째 read의 데이터)을 돌려줘 실패, 수정 뒤 20/20 순서대로



- **파형 (v1)**: rsp_ready가 풀리는 구간. 위 세 줄은 수정 전, 아래 두 줄은 수정 후. 수정 전에는 tag 4 자리에 17번째 read의 데이터(10)가 나가고 16건 뒤 rsp_valid가 내려간다. 수정 후에는 00부터 13까지 20건이 순서대로

v1 mem_ctrl.sv (diff)

```
- req_ready = free_found && !ref_due && !ref_active;
+ req_ready = free_found && !ref_due && !ref_active &&
+              (TAG_BITS'(tag_alloc + 1'b1) != tag_retire);
```

- **v4**: mc_resp의 tag_free가 같은 규칙. 랜덤 회귀의 'tag 한도에서 요청 대기' bin(1,192회 실행)이 이 상황을 매번 다시 만들고, HSL SMOKE도 칩에서 '요청 15개에서 ready 0'을 본다

배운 것

- 흐름 제어는 자원이 실제로 남는 곳(응답 버퍼)에서 해야 한다. 큐의 빈자리는 응답 버퍼의 여유를 뜻하지 않는다
- gap 목록에 적어 둔 상황을 재현 TB로 직접 만드는 것이 찾게 한 방법이었고, 그 TB를 회귀 시나리오로 남겼다

디버그 ②: 이슈 기록 — 증상, 원인, 조치 (v1 → v4, HSL 통합)

commit 이력에서 뽑은 버그 · 규칙 · 성능 · 타이밍 · 검증 · 통합 이슈 · v1 재검증 항목(B–E)이 어디서 닫혔는지 함께 적음

분류	증상	원인	조치 (commit)
v1 버그	back-pressure에서 응답이 틀리고 빠짐	tag wrap-around: 큐 빈자리만 보고 수락	진행 중 15개 한도 (fa08330, p.17)
v1 규칙	REF가 tREFI보다 늦음(밀린 최대 549), PREA 없이 REF	큐가 빌 때만 REF (drain-then-REF)	tREFI마다 빚지고 갚기 + PREA (v2 893c9a4), 체커가 위반으로 셈
v1 성능 (B)	closed RD 10사이클(v0 7)	큐 단계 +1, wait → issue handoff +1, 응답 버퍼 +1	명령 단위 스케줄러 (v2): 8사이클, 연속 읽기 3.4×(RD 4개)
v1 검증 (C)	뱅크 병렬성 확인이 v0에서도 통과	'열린 뱅크 ≥ 2'가 burst 전에 이미 참	동시에 일하는 뱅크 수로 판정 (7e2d4b5)
v1 검증 (D)	out-of-order · write-hit 경로 미도달	blocking task라 큐 점유가 최대 1	non-blocking 랜덤 TB: 재정렬 3,386,363 · write hit 1,102,895
v2 타이밍	age 비교가 151 MHz	tag 비교 체인	age matrix, 357 MHz (c1b8248, p.7)
v3 타이밍	첫 v3 208 MHz	쓰기 모드 → 후보 → pop → 같은 주소 비교 → req_ready, fanout 58 · 104	수락 경로 분리 · 후보 통합 · one-hot 유지 → 337 MHz (1b0943a)
v3 성능	쓰기 묶음이 처리량을 최대 12% 깎음	반대 큐를 아예 막음	우선순위만, 빈 슬롯은 반대 큐가 씀
v3 TB	포워딩 판정이 한 사이클 어긋남	엣지 카운터를 class 핸들로 읽음	엣지 번호를 인자로, RTL 그대로 (상세 p.23)
통합	ModelSim 10.1d 컴파일 실패	폭이 파라미터인 캐스트 21곳	타입 있는 localparam (44a0bb3)
통합	HSL 레인 BIST가 링크를 안 거칠 수 있음	쓰기 큐에 있는 주소의 읽기는 큐에서 답함	q_empty 출력 (44a0bb3)
열림 (E)	dram_model 오류 경로 미검증	컨트롤러가 그런 명령을 내지 않음	모델을 직접 모는 negative test

설계 trade-off: 선택마다 얻은 것과 낸 것

수치는 기본 빌드 회귀 · 포화 측정(commit ab9d522) · 기능을 하나씩 뺀 빌드의 처리량 · 면적 비교는 p.26

선택	이점	대가
명령 단위 스케줄러	요청마다 다음 명령을 바로 낸다. handoff가 없고 같은 row의 hit을 연달아 처리	매 사이클 16개 엔트리를 평가하는 조합 논리
행 · 열 명령 버스	PRE · ACT가 RD/WR와 슬롯을 다투지 않음. per-bank refresh와 함께 row hit 포화 +27%	명령 핀 두 벌, 같은 뱅크 PRE와 열 명령을 겹치지 않게 막는 논리
per-bank refresh	refresh 동안 다른 뱅크가 계속 일함. 밀린 refresh 최대 1개	refresh 명령이 4배, 뱅크를 하나씩 닫음
RFM (RAA 카운터)	RowHammer 한도를 지킴. 한도 도달 131,879회에도 위반 0	FF 20개, hotbank 처리량 약 5% (seed 3)
FR-FCFS와 row 보호	row hit을 먼저 처리해 hit rate와 처리량이 오름	오래된 conflict가 밀릴 수 있어 starvation 보호가 필요
읽기 · 쓰기 큐 분리	쓰기가 읽기 자리를 차지하지 않음. 읽기 큐는 데이터를 저장하지 않음	엔트리 16개, 두 큐 사이 순서(mc_xage, WDRAIN = 0일 때만 합성)
posted write + 포워딩	쓰기 응답 1사이클, RAW 읽기 37,434건이 DRAM 없이 응답(기본 빌드 회귀)	같은 주소 읽기가 큐에 있으면 쓰기를 포트에서 잡음
쓰기 모아 보내기	방향 전환 −19%, 읽기 latency −1.5%	처리량 이득은 없음 (FR-FCFS가 이미 묶음)
XOR 주소 매핑	v2 매핑보다 stride 3×, 연속 주소 +40% (v4 회귀)	연속 16 word가 한 뱅크 그룹 → 연속 읽기가 tCCD_L에 막힘
one-hot 선택	인덱스로 바꿨다 푸는 단계가 없음 (208 → 337 MHz의 일부)	데이터 선택이 AND-OR 트리
RD 4개 in flight	연속 읽기 3.4× (v1 대비)	read-return FIFO, PHY 왕복이 길면 부족 (p.20)
16 × 32b 응답 버퍼	호스트 계약이 단순함 (요청 순서)	FF의 34%

HSL 연결: 같은 RTL을 base die에 — DFI 형태 경계와 그쪽에서 나온 과제

High-Speed Link Lite가 이 저장소 44a0bb3의 RTL을 파일 그대로 씀(그쪽 0193c04) · HSL 숫자는 HSL commit 3d596a4, Nangate45 typical, 배선 없음

경계: 컨트롤러 신호가 HSL에서 가는 곳

컨트롤러	HSL
dram_rcmd · rbank · row	CA 레지스터 → pin_rca {cmd, bank, row}, 한 클럭 뒤
dram_ccmd · cbank · col · ap	pin_cca {cmd, bank, col, ap}
dram_wdata_en · wdata (WL 1)	DBI → 레인 shift → 동기 2:1 TX 기어 → 1.6 GHz beat
dram_data_valid · rdata	RDQS 캡처 → Gray FIFO → ECC 디코더 → rddata_valid. 지연이 고정되지 않아도 순서만 맞으면 됨
q_empty	레인 BIST가 쓰기 16개 뒤 큐가 빌 때까지 기다렸다 읽음
req_* · rsp_*	MISSION이고 phy_en일 때만 열리는 요청 포트. 시험 엔진이 돌면 엔진 차지

통합하며 이 저장소에서 고친 것 (44a0bb3)

- ModelSim-Altera 10.1d가 폭이 파라미터인 캐스트(BANK_BITS'(b) 등) 21곳을 읽지 못함 → 타입 있는 localparam, 회귀를 다시 돌린 뒤 가져감
- q_empty 출력: 쓰기 큐에 남은 주소의 읽기는 큐에서 답해 링크를 거치지 않기 때문

HSL이 짚은 과제

- 800 MHz 목표에서 최소 주기 2.18 ns(458 MHz). 한 클럭에 명령을 고르고 모든 뱅크 타이머를 갱신하는 루프(약 35단) → DFI 1 : 2(400 MHz, 클럭마다 명령 슬롯 둘)
- 읽기 대역: RD 하나가 낸 클럭부터 data_valid까지 슬롯을 잡는다. 이 저장소 모델은 3클럭, HSL PHY는 8클럭(CA 레지스터 1 + 핀 RD → rddata_valid 7). RD_PIPE 4라 읽기 32개가 97클럭 → RD_PIPE를 PHY 왕복에 맞춤
- 호스트 입력 → req_ready가 조합(포워딩 비교, 읽기 큐 충돌) −0.526 ns → 호스트 경계 레지스터 슬라이스

같은 RTL, 두 합성 흐름

흐름	최소 주기	매핑
이 저장소 make ppa	2.874 ns (348 MHz)	mem_ctrl만 flatten, 타이밍 목표 · 팬아웃 버퍼 없음
HSL 칩 STA	2.18 ns (458 MHz)	칩 전체를 경계 유지, abc -D 1250, 팬아웃 버퍼 트리

- **둘 다 Nangate45 typical, 배선 없음**. 어느 차이가 몇 %인지는 나눠 재지 않았다

버전별 latency: 구조가 바뀔 때 경로마다 몇 사이클이 오갔나

호스트 수락부터 응답 handshake까지(rsp_ready 1, directed TB에서 요청 하나씩) · v0 64e73e0, v1 2eabd33, v2 c1b8248, v3 1b0943a, v4 ab9d522

경로 (사이클)	v0	v1	v2	v3	v4	바뀐 이유
page hit RD	4	6	5	5	5	v1 큐 · 뱅크 엔진 단계, v2 명령 단위 스케줄러
closed-bank RD	7	10	8	8	8	v1 +3 (아래 분해)
row-conflict RD	11	14	11	11	11	
WR (뱅크 상태 무관)	6–10	9–14	6–9	1	1	v3부터 posted write
쓰기 큐에 있는 주소의 RD	–	–	–	1	1	v3부터 포워딩
refresh가 끼어든 읽기				23	5	v4는 refresh가 그 뱅크만 막음

closed-bank RD 분해 (v0 → v1, 사이클)

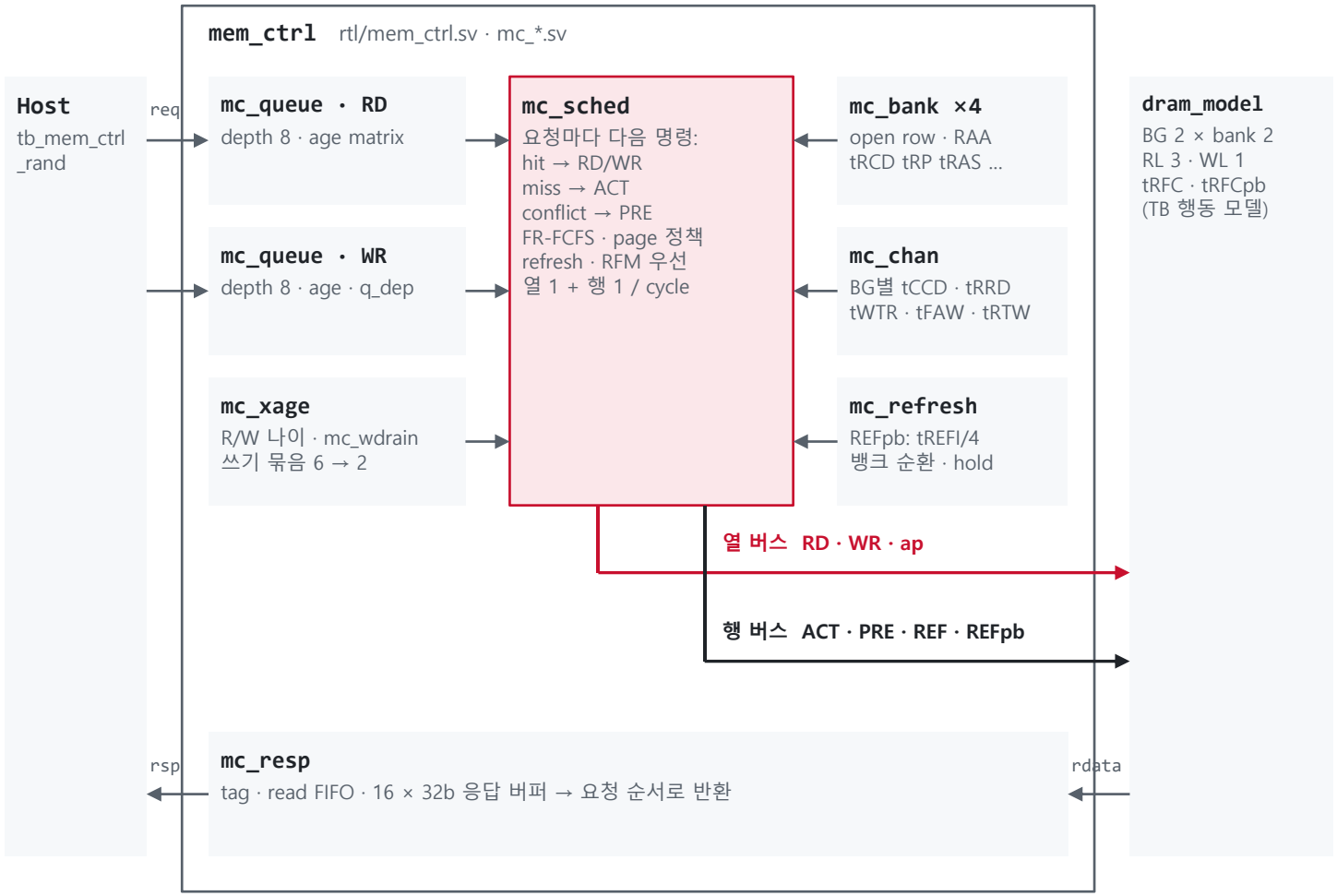
구간	v0	v1	v1에서 늘어난 이유
수락 → ACT	1	2	큐 단계 +1
ACT → RD	3	4	wait → issue handoff +1
RD → 응답	3	4	응답 버퍼 +1

읽는 법

- v0 단일 FSM은 요청 하나라 latency가 가장 짧지만 처리량이 없다: 4-bank burst 35클럭, v1은 17클럭(2.06×)
- v1은 뱅크 병렬을 얻고 경로마다 2–3사이클을 냈다. v2가 handoff를 없애고 RD를 4개까지 띄웠다
- v3 · v4는 경로 latency를 지키며 쓰기 · RAW 읽기 · refresh 영향을 줄였다. 처리량 비교는 p.24

v4 구조: 두 요청 큐 → 명령 단위 스케줄러 → 행 · 열 명령 버스

요청은 RD/WR가 나갈 때까지 큐에 머물고, 스케줄러가 매 사이클 요청마다 다음 명령을 계산해 열 버스와 행 버스에 하나씩 발행



한 사이클의 결정 (요청마다)

요청 상태	다음 명령	준비 조건
hit **row 열림**	RD / WR	tRCD, 그룹별 tCCD · tWTR(RD) · tRTW(WR), 같은 주소의 더 오래된 쓰기 없음
miss **뱅크 닫힘**	ACT	tRP · tRC · tRFCpb, 그룹별 tRRD, tFAW, RAA < 12
conflict **다른 row**	PRE	tRAS · tRTP · tWR, 그 row를 쓸 요청이 두 큐에 없음

- **열 버스**: 쓰기 묶음 모드가 고른 큐의 RD/WR, 없으면 다른 큐의 RD/WR 중 가장 오래된 것
- **행 버스**: refresh(PRE → REFpb) > RFM > 고른 큐의 ACT/PRE > 다른 큐의 ACT · 쓰지 않는 row PRE > timer PRE
- **가장 오래된 것**은 age matrix에서 AND-OR 한 단. 고른 결과는 one-hot 그대로 큐 pop · tag · 데이터 선택까지 간다
- **posted write · 포워딩**: 쓰기는 수락 엣지에 응답, 쓰기 큐에 같은 주소가 있는 읽기는 DRAM 없이 응답

검증 인프라: 독립 bind 체커 + 랜덤 TB, 회귀 1,200회 위반 0

체커는 RTL과 TB를 고치지 않고 DRAM 핀만 보고 붙는다 · 같은 TB가 기본 빌드와 파라미터 변형 15개를 모두 돈다 (commit ab9d522)

DRAM 타이밍 · 프로토콜 체커 (bind)

두 명령 버스만 보고 open row · 명령 시각 · 뱅크별 refresh · RAA를 따로 재구성해 **타이밍 18종**(뱅크 그룹 S/L, tRFCpb 포함)과 **프로토콜 10종**(RAA 한도 포함)을 검사. auto-precharge는 장치가 precharge하는 엣지를 계산해 다음 ACT의 tRP를 잰다. mem_ctrl 내부 신호는 읽지 않는다.

랜덤 TB (UVM 구조를 본뜬 class TB)

sequence → driver → DUT → monitor → scoreboard(참조 메모리). 주소 매핑을 RTL과 따로 구현하고, 열 버스에 없는 row는 행 버스의 ACT로 추적. posted write는 WR가 실제로 나가는지, 포워딩은 RD가 나가지 않는지 핀에서 확인. **기능 bin 32개**, seed 재현.

호스트 SVA · directed 시나리오

valid/ready concurrent assertion 4개 + cover 5개. directed 6개: hit, miss, conflict, 4-bank burst, 응답 back-pressure(tag 16개 초과), 포워딩.

디버그: 포워딩 판정 78회 중 77회 FAIL (v3)

bind probe로 cycle 265 확인. 쓰기의 WR가 나가는 엣지에 같은 주소 읽기가 수락되는 경우를, 핀 모니터가 같은 엣지에 갱신되는 카운터를 class 핸들(vif.cyc)로 읽어 판정이 흔들린 **TB race** → TB top이 엣지 번호를 인자로 전달. RTL 변경 없이 960/960 PASS.

랜덤 회귀 (make regress)

빌드	실행	PASS	위반
기본: 두 버스 · per-bank · RFM · 쓰기 묶음 · open · XOR	300	300/300	0
refresh · 버스: bus1 · refab · bus1_refab · rfm_late	240	240/240	0
스케줄링: fcfs · fcfs_nodrain · nodrain · starve24	240	240/240	0
구조: rdpipe1 · depth4	120	120/120	0
page · 매핑: close · timer · map_brc · map_rbc · map_rcb	300	300/300	0
합계 16 빌드 · 600만 요청	1,200	1,200/1,200	0

재정렬 3,386,363 · 포워딩 151,757 · 행 / 열 동시 발행 1,359,346 · RAA 한도 도달 131,879회. 응답 순서와 데이터 모두 일치.

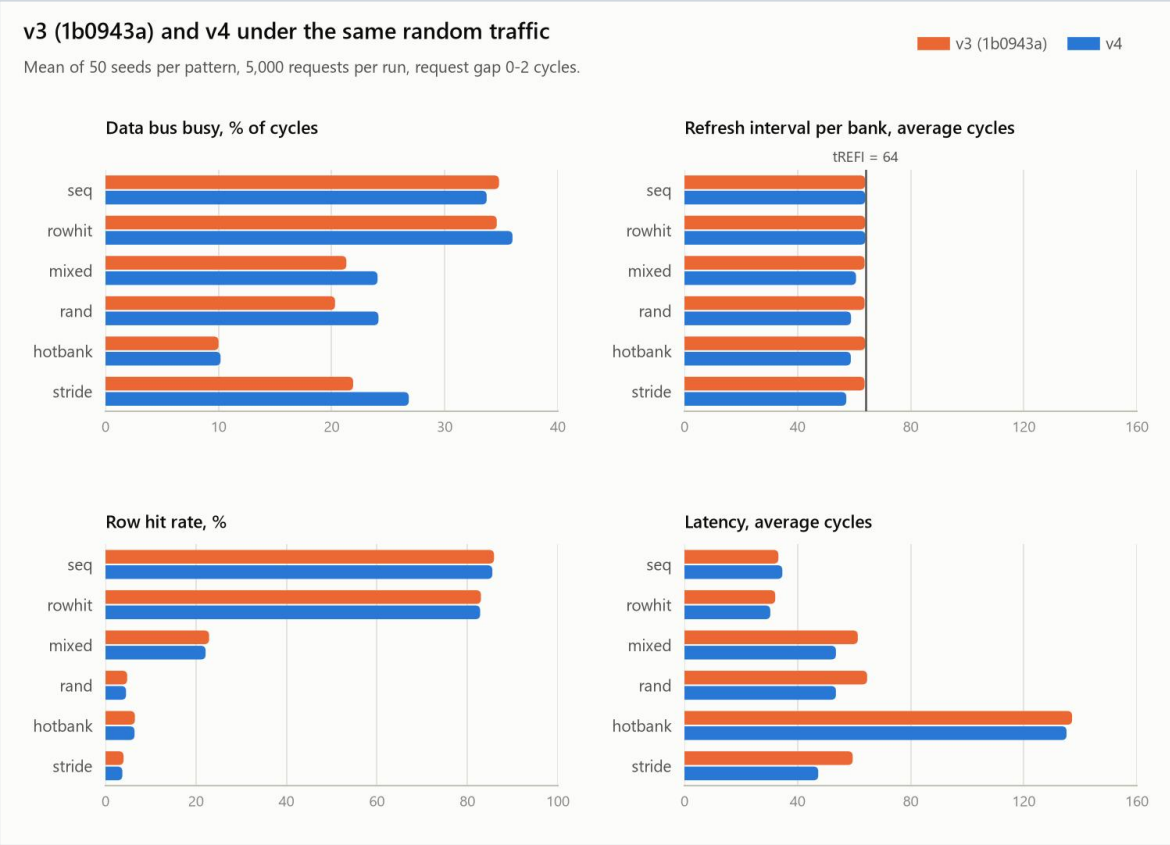
코드 커버리지 (make cov, directed → + 랜덤)

범위	Line	Branch	Toggle
컨트롤러 (mem_ctrl + mc_*)	94.8 → **98.7%**	81.8 → **98.2%**	48.5 → **93.1%**
mc_sched	91.9 → **100%**	62.5 → **96.9%**	43.2 → **91.5%**
RTL 전체 (dram_model 포함)	88.9 → **96.0%**	77.7 → **94.1%**	53.9 → **93.7%**

남은 구멍: older() 분기 2개(루프를 펼치며 상수로 접힘), 밀린 refresh 카운터 포화(설계상 미도달), WL > 1 쓰기 파이프 단, dram_model 오류 경로(negative test 필요).

성능: 같은 트래픽에서 v3 → v4, 포화 처리량을 기능별로 분해

랜덤 회귀 평균(seed 50 × 5,000건)과 포화(요청 간격 0, seed 1) · 표의 가운데 네 열은 v4를 파라미터로 바꾼 빌드



v3 = commit 1b0943a(뱅크 그룹 없는 장치), v4 = ab9d522. conflict가 많은 트래픽이 가장 많이 올랐다(rand +19%, stride +22%). seq만 −3%: 같은 그룹 안 연속 RD/WR가 tCCD_L(2)를 지켜야 한다. 뱅크당 refresh 간격 57–64(RFM의 REFpb 포함), 밀린 refresh 최대 1개.

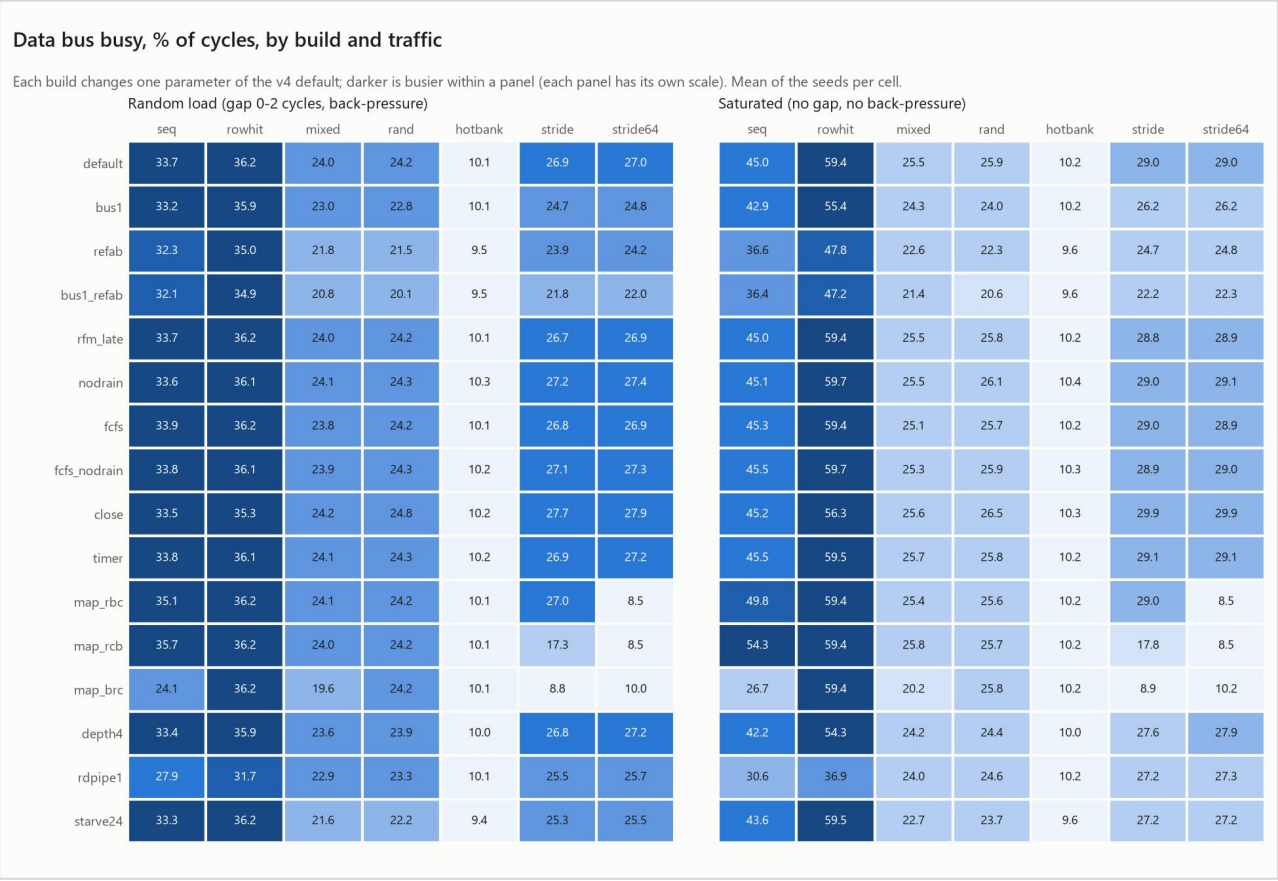
포화 처리량: 두 기능을 하나씩 (데이터 버스 사용률)

트래픽	v3 방식 bus1_refab	버스 분리만 refab	per-bank만 bus1	v4	v4 + row:col:bank
row hit, 읽기만	63.3%	65.6%	71.8%	**80.7%**	80.7%
row hit, 쓰기만	60.4%	62.1%	72.3%	**80.7%**	80.7%
row hit, 반반	47.3%	47.8%	55.2%	**59.6%**	59.6%
연속 주소, 읽기만	39.7%	40.2%	48.0%	**50.0%**	**78.5%**
랜덤 주소, 읽기만	21.9%	23.4%	24.7%	**26.5%**	27.1%
패턴 혼합, 반반	21.7%	22.7%	24.3%	**25.7%**	25.6%

- **per-bank refresh가 가장 크다**: row hit 읽기 63.3 → 71.8%. all-bank는 tREFI마다 모든 row를 닫고 약 25사이클 멈춘다
- **두 기능은 서로 키운다**: 버스 분리 단독은 +1–7%, per-bank와 함께면 71.8 → 80.7%. refresh의 PRE · REFpb가 행 버스로 가서 RD 흐름을 끊지 않는다
- **뱅크 그룹은 매핑을 바꾼다**: 연속 16 word가 한 그룹에 있는 XOR은 연속 읽기가 50%에서 막히고, 두 word마다 그룹을 바꾸는 row:col:bank는 78.5%
- **경로별 latency**: hit RD 5 · miss RD 8 · conflict RD 11사이클, posted WR와 포워딩된 RD는 1사이클 (v3부터)

트래픽 × 기능 분석: 빌드 16 × 트래픽 7 × 두 부하 = 1,456회

v4 기본에서 파라미터를 하나씩 바꾼 빌드를 같은 트래픽에 돌리고 같은 파라미터로 합성 (make matrix · ppa-sweep)



데이터 버스 사용률(%). 왼쪽 랜덤 부하(seed 10), 오른쪽 포화(seed 3). 1,456회 모두 PASS, 규칙 위반 0, 데이터 불일치 0 (RTL ab9d522, 스크립트 322632f).

질문별 답 (포화 평균 기준)

질문	답	근거
가장 큰 손잡이	**per-bank refresh**	all-bank로 바꾸면 –16%, 면적 차이 2%
명령 버스 분리	면적 비용 없음	하나로 바꾸면 –7%, 면적 0.4% 이내
주소 매핑	**XOR만** 모든 stride에서 버팀	64 word stride: XOR 27.0%, 나머지 8.5–10.0%
큐 8 + 8	처리량 +6%에 면적 +46%	4 + 4는 읽기 latency 절반, 면적 –31%
RD 파이프라이닝	필수	RD 1개로 줄이면 –19%, 면적 0.4%
FR-FCFS vs FCFS	이 모델에서는 같음	FCFS가 면적 +8% (두 큐 사이 나이 필요)
쓰기 모아 보내기	평균 ↓, 꼬리 ↑	읽기 평균 –1.7%, p99 +9%, 방향 전환 –19%
RFM 미리 주기	빼도 된다	성능 같음, 면적 –0.8%

fmax 차이는 ABC 잡음(±12%) 안이면 구조 차이로 보지 않는다: 비트 배선만 다른 세 매핑이 모두 310 MHz, XOR 게이트 두 개가 더 있는 기본 빌드가 348 MHz. FF 수는 정확하고 면적은 ±2–3% 안에서 믿을 만하다.

PPA: 기능은 늘고 fmax는 유지 — 버전별 면적 · 타이밍과 기능별 비용

sv2v → Yosys 0.52 → Nangate45 typical → OpenSTA 3.1 · mem_ctrl 단독(배선 · 클럭 트리 없음) · 버전마다 그 commit을 다시 합성

버전별 (make ppa)

	v1 fa08330	v2 c1b8248	v3 첫 구현 커밋 전	v3 1b0943a	v4 ab9d522
Area μm²	9,692	13,478	19,263	18,600	**18,912**
Cell	4,433	6,781	10,991	9,924	**9,995**
FF	1,072	1,344	1,582	1,582	**1,616**
reg2reg ns	2.880	2.753	4.765	2.922	**2.831**
≈ fmax MHz	343	357	208	337	**348**

v3 타이밍 클로저: 208 → 264 → 337 MHz

- **증상**: 쓰기 모드 → 후보 선택 → 큐 pop → 같은 주소 비교 → 포워딩 · req_ready → push. fanout 58 · 104 넷 두 개가 2 ns
- **① 수락 경로 분리**: 같은 주소 비교가 이번 엣지에 나가는 엔트리도 그대로 비교 → 스케줄러 결정을 기다리지 않음
- **② 후보 집합 통합**: "고른 큐, 없으면 다른 큐" 한 집합씩으로 가장 오래된 것 고르기 5 → 3번, pop 때 의존 비트 지우기 제거
- **③ one-hot 유지**: 인덱스로 바꿨다 다시 푸는 단계(fanout 72) 제거, AND-OR 선택
- **v4**: 두 버스라 RD/WR와 ACT/PRE 중 고르는 단계가 사라짐 → RFM을 넣고도 348 MHz (RFM 전 377 MHz)

기능 하나를 뺐을 때 (포화 평균 · 랜덤 부하 읽기 latency)

기능 (대신)	빼면 처리량	빼면 latency	이 기능의 면적	판단
per-bank refresh (all-bank)	−16%	+10%	+2%	가장 싸고 가장 큼
두 명령 버스 (하나)	−7%	+4%	~0%	공짜에 가까움
RD 4개 in flight (1개)	−19%	+8%	+0.4%	필수
큐 8 + 8 (4 + 4)	−6%	−45%	+46%	처리량 우선일 때만
XOR 매핑 (row:bank:col)	stride64 −69%	+28%	잡음 범위	기본 유지
FR-FCFS (FCFS)	0%	0%	−8%	이 모델에선 면적 이득
쓰기 묶음 (오래된 순)	0%	+2%	−2%	평균 · 전환 vs p99
미리 RFM (한도에서만)	0%	0%	+0.8%	빼도 된다

"빼면"은 기본 빌드 대비 그 변형 빌드의 변화, 면적은 그 기능이 기본 빌드에 더하는 양이다. fmax는 잡음(±12%)보다 큰 차이만 의미가 있다(depth4 +18%, timer −23%, fcfs_nodrain −24%). 권장: 범용은 지금의 기본값, 면적 · latency 우선이면 큐 4 + 4(처리량 −6%, 면적 −31%). ≈ fmax = 1 / 최소 주기(reg2reg + 플롭 setup, v4는 2.831 + 0.043 = 2.874 ns).

12 Conclusion

핵심 요약 및 향후 계획

Spec → RTL → 검증 → 성능 / PPA 분석 전 과정 결과와 미해결 항목 정리

핵심 요약

같은 DRAM 모델에서 컨트롤러를 다섯 번 설계하며 전환마다 성능과 비용을 수치로 비교하고,
독립 체커와 랜덤 회귀(1,200회, 위반 0)로 단계마다 정확성을 확인

향후 계획

- 뱅크 그룹을 word 단위로 섞는 XOR 매핑 (연속 주소와 stride를 함께)
- 배치 · 배선 흐름(OpenROAD)으로 기능별 타이밍 비용 재측정
- 응답 버퍼(FF 34%) 축소, dram_model 오류 경로 negative test
- DFI 1 : 2(400 MHz, 클럭당 명령 2개)와 RD_PIPE를 PHY 왕복에 맞추기 (HSL 연결에서 나온 과제)
- 실제 장치 규모(pseudo channel · 뱅크 16개 이상)에서 refresh · RFM 비중 재평가

권용현

010-6408-8646

kyongh0331@naver.com

github.com/yhkwon-git/mini-dram-controller